

第22讲：Chain Rule

一个参数怎样通过很多中间节点影响最终 Loss?

核心概念：【必须真正掌握】

本讲只学习 Backpropagation 真正需要的 Chain Rule。目标不是做高等数学证明，而是看清：一个早期变量怎样通过多个中间节点影响最终 Loss，以及为什么这些“局部导数”要沿路径相乘。



建议学习方式：先手算单链 -> 再看神经元链 -> 运行数值差分 -> 最后理解分支与 Residual。

一、本讲学习目标

理解 Chain Rule（链式法则）解决的不是“一个函数怎么求导”，而是“变量通过多层中间关系间接影响最终结果时，影响怎样传递”。

能手算 $x \rightarrow y \rightarrow L$ 、 $x \rightarrow a \rightarrow b \rightarrow L$ 两类最小链式例子。

理解“局部导数相乘”为什么给出整条路径的总敏感度。

理解神经元 $w \rightarrow z \rightarrow \text{activation} \rightarrow \text{Loss}$ 中参数梯度是怎样被逐段传回来的。

理解 ReLU 的局部导数可以像“门”一样让梯度通过或归零。

理解当一个变量通过多条路径影响 Loss 时，各路径贡献需要相加。

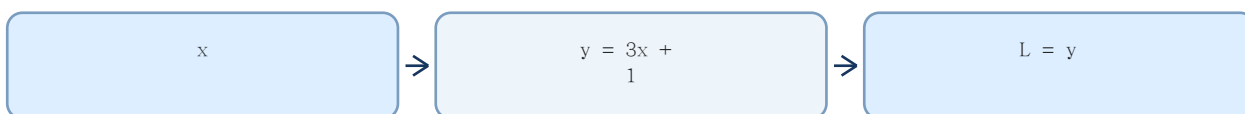
把 Chain Rule 与 Transformer、Residual Connection、Backpropagation 建立最低必要联系。

核心概念：与第19-21讲的关系

偏导数告诉你单个参数对 Loss 的局部敏感度；Gradient 把所有偏导数组成方向；但真实网络的 Loss 往往通过很多中间节点间接依赖参数。Chain Rule 正是把“中间节点的局部敏感度”串起来。

二、【原理】为什么需要 Chain Rule?

先看一个最简单的间接关系：



这里 Loss 并没有直接写成“某个复杂网络对 x 的函数”，而是： x 先影响 y ， y 再影响 L 。我们真正想知道的是：

$$dL/dx = ?$$

Chain Rule 告诉我们：

$$dL/dx = (dL/dy) \times (dy/dx)$$

具体例子：通俗解释

如果 x 改 1 个小单位，会先让 y 改 dy/dx 倍；而 y 的这点变化又会让 L 改 dL/dy 倍。两段敏感度连续发生，所以总影响是乘起来。

数学直觉：不要把它记成死公式

更重要的阅读方法是：沿计算路径从 Loss 往前找，每经过一条边，就乘上这一小段的局部导数。

三、【具体数字例子】两段链手算

$$y = 3x + 1$$

$$L = y^2$$

取 $x=2$:

步骤	数值
x	2
$y=3 \times 2+1$	7
$L=7^2$	49

现在逐段求局部导数:

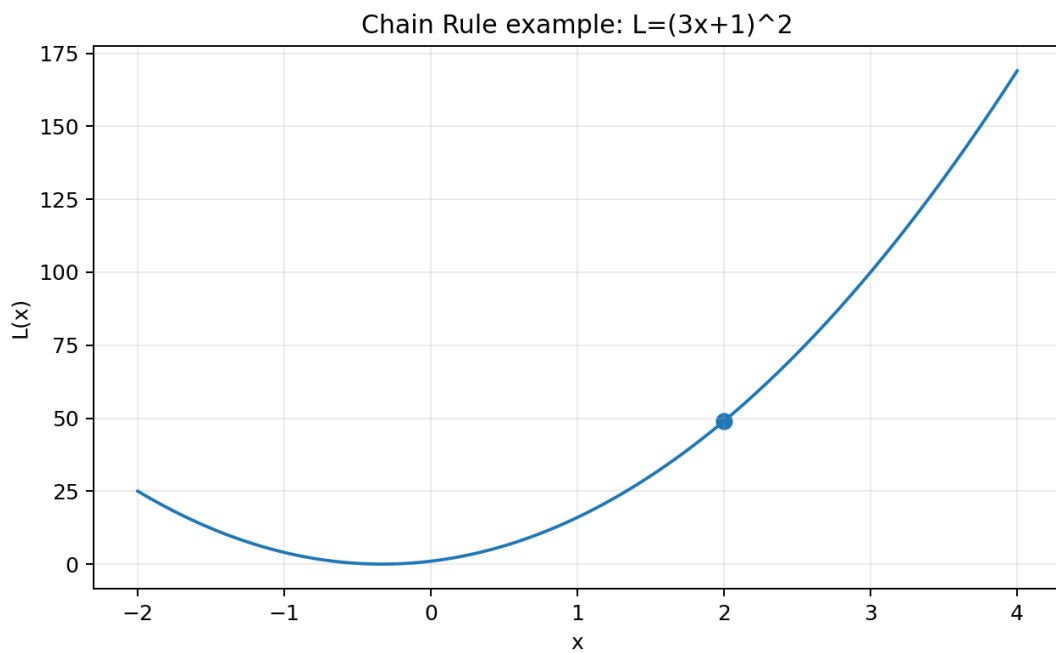
$$dL/dy = 2y = 14$$

$$dy/dx = 3$$

所以:

$$dL/dx = 14 \times 3 = 42$$

这句话可以直接读成: 在 $x=2$ 附近, x 增加很小的 Δx 时, Loss 大约变化 $42 \times \Delta x$ 。



四、【公式逐项拆解】每一项究竟是什么意思？

符号	读法	真正含义
dL/dy	L 对 y 的导数	y 稍微变一点，L 会怎样变
dy/dx	y 对 x 的导数	x 稍微变一点，y 会怎样变
dL/dx	L 对 x 的总导数	x 最终通过 y 对 L 产生多大影响
$\times$	相乘	两段局部敏感度沿同一路径连续传递

具体例子：类比：齿轮传动

第一组齿轮把输入变化放大 3 倍，第二组再放大 14 倍，那么从最前端到最末端的局部传动倍率就是 $3 \times 14 = 42$ 。

数学上，Chain Rule 并不是把“单位”真的约掉；但把 $dL/dy \times dy/dx$ 看成中间变量 y 被连接起来，是很有用的直觉。

五、【更长的链】局部导数一路相乘



取 $x=2$:

$$x=2 \rightarrow a=4 \rightarrow b=5 \rightarrow L=25$$

局部导数:

$$dL/db = 2b = 10$$

$$db/da = 1$$

$$da/dx = 2$$

于是:

$$dL/dx = (dL/db)(db/da)(da/dx) = 10 \times 1 \times 2 = 20$$

核心概念: 这就是 Backprop 的最小雏形

Forward 时从 x 一路算到 L ; Backward 时从 L 反向沿依赖关系, 把每一段局部导数乘起来。网络再深, 本质思想不变。

六、【神经元例子】Weight 的梯度怎样穿过 ReLU?

构造最小神经元链:

$$z = wx + b$$

$$a = \text{ReLU}(z)$$

$$L = (a - \text{target})^2$$



取: $x=2$, $w=1.5$, $b=-1$, $\text{target}=4$ 。Forward:

$$z=2, a=2, L=4$$

Backward:

$$dL/da = 2(a - \text{target}) = -4$$

$$da/dz = 1 \quad (\text{因为 } z > 0)$$

$$dz/dw = x = 2$$

所以:

$$dL/dw = (-4) \times 1 \times 2 = -8$$

Bias 同理: $dz/db=1$, 因此 $dL/db=-4$ 。

数学直觉: 如何解释负号

$dL/dw=-8$ 表示在当前点附近, 如果 w 稍微增大, Loss 倾向下降; 如果 w 稍微减小, Loss 倾向上升。

七、【ReLU 像一扇梯度门】

当 $z < 0$ 时, $\text{ReLU}(z)=0$, 并且在普通实现中这一侧的局部导数是 0:

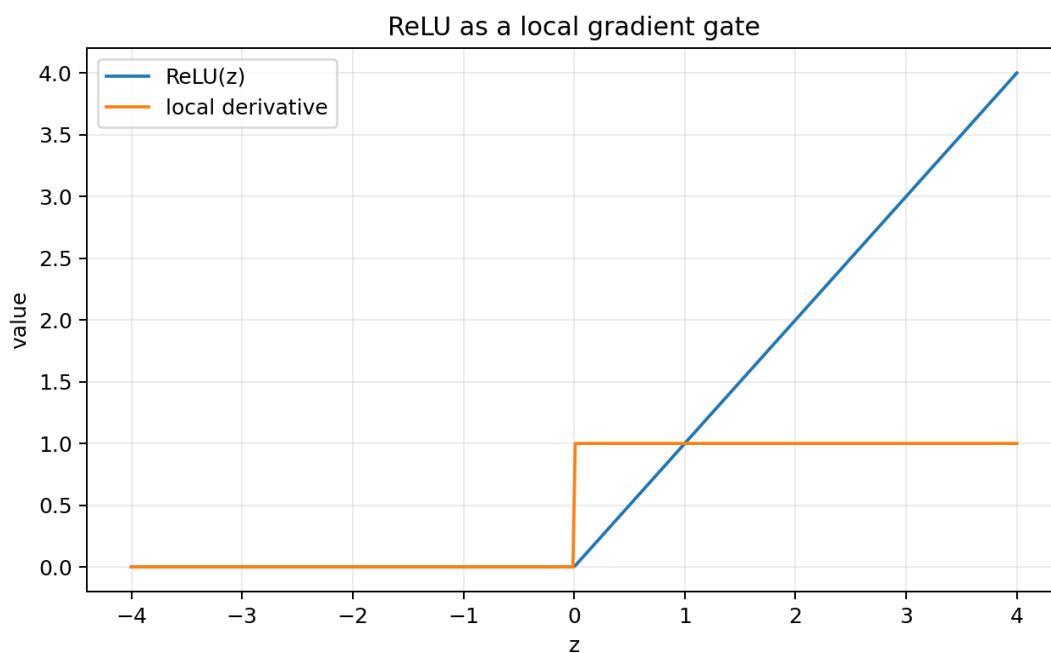
$$d \text{ReLU}(z) / dz = 0 \quad (z < 0)$$

于是整条链:

$$dL/dw = (dL/da) \times (da/dz) \times (dz/dw)$$

中间只要乘到 0:

$$dL/dw = \dots \times 0 \times \dots = 0$$



常见误区 / 风险: 不要过度简化

这不是说“ReLU 一定让神经元永远学不了”。参数和输入会变化, 其他样本也可能让 $z > 0$ 。但如果一个 ReLU 单元长期处在负区间, 确实可能出现梯度长期为0的 dying ReLU 问题。

八、【分支计算图】多条路径为什么要相加？

Chain Rule 不只处理单链。假设同一个 x 同时走两条路径：

$$u=x^2$$

$$v=3x$$

$$L=u+v$$

x 对 L 的影响有两条路径：

路径	局部乘积	在 $x=2$ 时贡献
$x \rightarrow u \rightarrow L$	$(dL/du)(du/dx)$	$1 \times 4=4$
$x \rightarrow v \rightarrow L$	$(dL/dv)(dv/dx)$	$1 \times 3=3$

总影响要把两条路径相加：

$$dL/dx = 4 + 3 = 7$$

科研连接：为什么这和 Residual 很重要

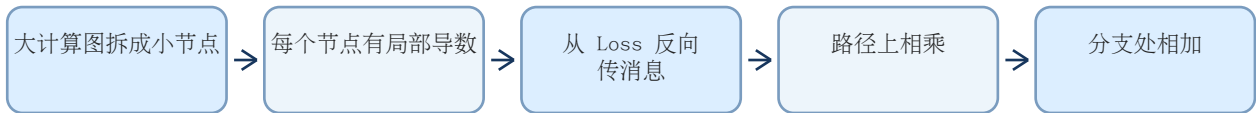
如果 $y=F(x)+x$ ，那么 x 一方面通过 $F(x)$ 影响后续结果，另一方面通过 identity shortcut 直接影响后续结果。反向时，两条路径的梯度贡献会在分叉点汇合并相加。

九、【Computational Graph 视角】局部规则如何复用？

对任何计算节点，都可以只关心两件事：

阶段	节点需要做什么
Forward	根据输入计算自己的输出，并保存必要中间量
Backward	接收“后面传来的梯度”，乘本节点局部导数，再把结果传给前面的输入

所以 Backpropagation 并不是为每个巨大网络重新推一个超级公式，而是：



核心概念：最低闭环理解

Backpropagation = 在计算图上高效、系统地重复应用 Chain Rule。

十、【数值差分验证】我们怎样知道手算没错？

第13讲学过数值导数。对参数 w ，可以用：

$$dL/dw \approx [L(w+\epsilon)-L(w-\epsilon)] / (2\epsilon)$$

本讲神经元实验中：

量	Chain Rule 解析结果	数值差分
dL/dw	-8.000000	-8.000000
dL/db	-4.000000	-4.000000

这就是以后所谓 `gradient check` 的最小思想：用慢但直接的数值差分，抽查反向传播公式或实现是否正确。

科研连接：科研用途

你不需要用数值差分训练模型，但在自己实现新模块、新 Loss 或自定义 backward 时，`gradient check` 是排查数学与代码错误的经典工具。

十一、【深网络预览】为什么局部导数连乘值得警惕？

如果连续很多层的局部导数都小于 1，例如每层约 0.5，那么：

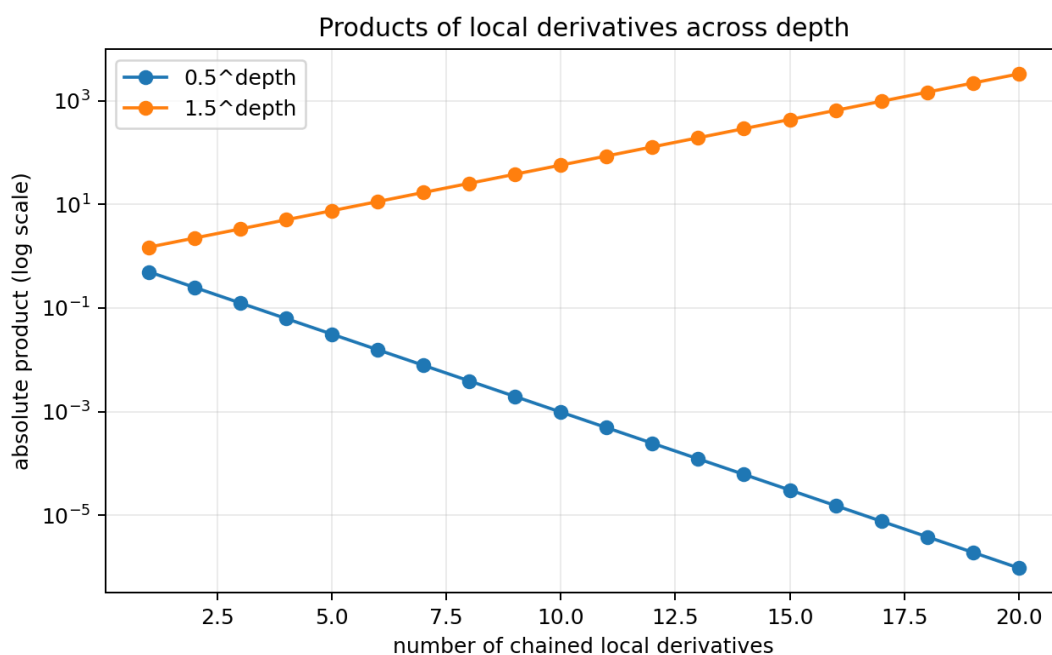
$$0.5^{10} \approx 0.00098$$

$$0.5^{20} \approx 0.00000095$$

如果连续很多层的局部导数大于 1，例如每层约 1.5：

$$1.5^{10} \approx 57.67$$

$$1.5^{20} \approx 3325.26$$



数学直觉：这里只做预告

这不是 Vanishing/Exploding Gradient 的完整理论，但已经能看出：Backprop 中大量局部导数的乘积，可能让早期层收到的梯度非常小或非常大。后面会专门学习。

十二、【与 Transformer / LLM 的关系】

LLM 组件	Chain Rule 在做什么
Linear / QKV Projection	Loss 通过矩阵乘法把梯度传回 Weight 和输入
GELU / SiLU	把后方梯度乘以激活函数局部导数
Attention	Loss 通过 score、Softmax、加权求和等多个节点反传
Residual Connection	梯度通过主分支与 shortcut 两条路径回传并汇合
Cross Entropy	从最终 logits 产生最末端的梯度起点
Embedding	最终梯度会一路传到被访问的 embedding 参数

科研连接：你以前在 LLM 课里见过 Backpropagation

现在可以重新理解：LLM 预训练不是“Loss 自动神秘地传回几十亿参数”，而是庞大计算图里的每个运算节点都提供自己的局部导数规则，整个系统用 Chain Rule 把这些局部规则组合起来。

十三、【常见误区】

常见误区 / 风险：误区1：Chain Rule 就是背 $dL/dx = (dL/dy)(dy/dx)$

公式只是结果；真正能力是能沿计算图识别中间变量和每一段局部导数。

常见误区 / 风险：误区2：所有梯度只需要相乘

单条路径上相乘；如果一个变量通过多条路径影响 Loss，路径贡献要相加。

常见误区 / 风险：误区3：Backward 是把 Forward 的算式倒着重新算一遍

Backward 依赖 Forward 的计算图和中间值，但执行的是局部导数和梯度传播，不是简单倒放 Forward。

常见误区 / 风险：误区4：局部导数为0就代表整个网络所有参数梯度都为0

只会阻断经过该局部节点的相应路径；网络其他路径仍可能有梯度。

常见误区 / 风险：误区5：Chain Rule 只适用于标量网络

标量例子最容易学；向量/矩阵网络仍遵循同样的复合函数思想，只是梯度对象和维度更丰富。

常见误区 / 风险：误区6：只要会自动求导，就不需要懂 Chain Rule

看训练异常、设计新模块、读公式、理解 detach/stop-gradient/Residual 时，都需要知道梯度路径是怎样组成的。

十四、【代码实验】本讲实验做了什么？

实验	问题	验证结果
1. 两段链	局部导数相乘是否得到总导数？	42，与数值差分一致
2. 长链	三段局部导数能否继续相乘？	得到20
3. 神经元链	Weight 梯度怎样穿过 ReLU？	$dL/dw=-8$
4. ReLU 关闭	局部导数0会发生什么？	该路径梯度归0
5. 分支图	两条路径怎样合并？	$4+3=7$
6. 深链预览	连乘会不会缩小/放大？	0.5^n 快速衰减， 1.5^n 快速放大

建议你运行代码时，不要只看最终答案。把每一个局部导数打印出来，再自己按路径乘一次。

```
# 最小 Chain Rule
x = 2.0
y = 3*x + 1
L = y**2

dL_dy = 2*y
dy_dx = 3.0
dL_dx = dL_dy * dy_dx
print(dL_dx) # 42
```

十五、本讲知识闭环



把第19-22讲连起来：

Partial Derivative -> Gradient -> Loss Landscape -> Chain Rule

下一讲我们会更进一步，不再只看几条手写链，而是把一个完整计算图拆成节点，逐个写出局部梯度，然后执行一次真正的手工 backward。

核心概念：下一讲

第23讲：Computational Graph 与局部梯度——把 Backpropagation 拆成可执行的节点规则。